Università degli Studi di Messina
Dipartimento MIFT - Corso di Laurea Triennale in Informatica (L-31)

Laboratorio di Reti e Sistemi Distribuiti

HANDS-ON 8:
Multithreading e Thread Affinity

Massimo Mantineo
Matricola: 541924

Messina, A.A. 2024/2025

Indice

1 Problem Statement

L'ottava esercitazione (HO8) affronta il tema del calcolo parallelo e dell'ottimizzazione del carico di lavoro su architetture multi-core. L'obiettivo consiste nella ricerca dei numeri primi in un intervallo esteso ($1 - 1.000.000$) tramite l'impiego della libreria **Pthreads**. La sfida tecnica non risiede solo nella parallelizzazione algoritmica, ma nell'esplorazione della **Thread Affinity**, una tecnica avanzata che permette di vincolare i flussi di esecuzione a specifici core fisici per massimizzare il riuso della cache e minimizzare l'overhead dello scheduler di sistema.

2 Stato dell'Arte

L'efficienza del multithreading moderno dipende strettamente dall'interazione tra lo scheduler del sistema operativo e la topologia hardware.

- **SMT e Hyper-Threading**: Molti processori moderni presentano core logici che condividono le unità di esecuzione fisiche. Senza affinità, lo scheduler potrebbe posizionare due thread pesanti sullo stesso core fisico, dimezzando le prestazioni.
- **Gerarchia delle Cache e NUMA**: L'affinità hardware (*Hard Affinity*) è cruciale per mantenere i dati nelle cache L1/L2 locali al core, evitando costosi accessi alla memoria principale (`RAM`) causati dalla migrazione dei thread tra core distanti.

- **Bitmasking (`cpu`$_set_t$)** : $SottoLinux, l'affinitàvienegestitatramitemascheredibitcherappresentanolar$

3 Metodologia ed Implementazione

L'implementazione prevede la suddivisione dell'intervallo di ricerca e l'assegnazione deterministica dei core.

3.1 Parallelizzazione Alternata

Il carico viene distribuito equamente tramite una struttura `thread`$_data_t, garantendocheognithreadproces$ $intervallounico$(`Static Partitioning`).

3.2 Algoritmo di Verifica

Viene utilizzato un algoritmo di test di primalità ottimizzato (fino alla radice quadrata di N) per ridurre la complessità computazionale di ogni singolo thread.

4 Risultati

La validazione è stata condotta monitorando la distribuzione del carico in tempo reale.

4.1 Validazione dell'Affinità (PSR)

Tramite il comando `ps -eLo pid,tid,psr`, è stato verificato che il thread principale e i thread worker occupassero core distinti senza fluttuazioni.

Monitoraggio PSR

```
  PID   TID PSR CMD
20164 20165   0 ./prime_affinity (Thread 0)
20164 20166   1 ./prime_affinity (Thread 1)
```

4.2 Vantaggi Prestazionali

I test hanno confermato che forzare l'affinità riduce la varianza dei tempi di esecuzione (*jitter*), rendendo il sistema più prevedibile per carichi di calcolo intensivo.

5 Conclusioni e Sviluppi Futuri

L'Hands-On 8 ha evidenziato come la vicinanza tra software e hardware sia determinante per le prestazioni. Il controllo dell'affinità trasforma lo scheduler da una "black box" a uno strumento deterministico.
Sviluppi futuri:

- **Work-Stealing**: Implementazione di uno scheduler dinamico che ripartisce il carico se un thread completa il proprio intervallo più velocemente degli altri.
- **OpenMP**: Migrazione verso direttive di alto livello come OpenMP per gestire la parallelizzazione e l'affinità in modo dichiarativo e portabile.

mputing).